Facultad de Ciencias Exactas, Físicas y Naturales - Universidad Nacional de Córdoba

Ingeniería en Computación

Trabajo Práctico N°2: UART

Maquinas de Estado Finitas: UART

Arquitectura de Computadoras

Integrantes

Brigido, Tomás Andrés (Legajo 38481096)
Tosolini, Agustín (Legajo 42332698)

Docente

Pereyra, Martin

Repositorio

https://github.com/Agustin-Tosolini/Computer-Architecture-FCEFyN/tree/main/Laboratories/Lab2

Contents

1 Introducción **4**

1.1 Herramientas utilizadas 4

2 Marco teórico **5**

2.1 Máquinas de estado finitas 5

2.1.1 FSMD 5

2.1.2 Codificación de estados 5

2.2 Comunicación serie asíncrona 6

2.2.1 Sobremuestreo 16x 6

3 Arquitectura del sistema **7**

3.1 Esquemático 7

4 Generador de baud rate **10**

5 Receptor **11**

5.1 Máquina de estados (`rx_fsmd`) 11

5.1.1 Sincronizador de entrada 12

5.1.2 Elección del punto de muestreo 12

5.2 Interface del receptor (`rx_intf`) 12

6 Transmisor **14**

6.1 Máquina de estados (`tx_fsmd`) 14

6.2 Interface del transmisor (`tx_intf`) 15

7 ALU y control del protocolo **16**

7.1 Protocolo 16

7.2 Máquina de control (`ALU_ctrl`) 16

7.3 ALU 17

8 Verificación **18**

8.1 Loopback del transmisor y el receptor 18

8.2 Sistema completo 19

9 Implementación en la placa **21**

9.1 Conexión física 21

9.2 Asignación de pines 21

9.3 Prueba con la PC 21

10 Conclusiones **23**

1 Introducción

El objetivo de este trabajo práctico es diseñar e implementar en Verilog una UART (*Universal Asynchronous Receiver and Transmitter*) y utilizarla para comunicar la ALU desarrollada en el trabajo práctico anterior con una computadora.

El sistema funciona bajo un esquema de pregunta y respuesta: la PC envía por el puerto serie tres bytes (el operando A, el operando B y el código de operación), la FPGA los recibe, la ALU calcula el resultado y este vuelve a la PC como un único byte. Las banderas de la ALU (*zero*, *carry* y *overflow*) se muestran en los LEDs de la placa.

El trabajo se apoya casi por completo en máquinas de estado finitas: el receptor, el transmisor y el control del protocolo son tres máquinas independientes que se coordinan entre sí mediante señales de *handshake*.

1.1 Herramientas utilizadas

- **Lenguaje:** Verilog (únicamente).
- **Síntesis e implementación:** Vivado 2026.1.
- **Placa:** Digilent Basys 3 (Artix-7 XC7A35T-1CPG236C, reloj de 100 MHz).
- **Verificación:** simulación funcional con dos *testbenches* propios.
- **Prueba sobre la placa:** script de Python con interfaz gráfica (Tkinter y `pyserial`) que envía los operandos y verifica el resultado contra un modelo de la ALU escrito en Python.

2 Marco teórico

2.1 Máquinas de estado finitas

Una máquina de estado finita (FSM) modela un sistema que en cada instante se encuentra en *un único* estado de un conjunto finito, y que evoluciona según la secuencia de entradas que recibe. Son la herramienta natural para problemas en los que la acción a tomar depende de lo que ocurrió antes, y no solo de la entrada actual.

Según de qué dependan las salidas se distinguen dos tipos:

- **Moore:** la salida depende únicamente del estado actual. Como proviene directamente de registros, no presenta *glitches*.
- **Mealy:** la salida depende del estado y de la entrada. Suele requerir menos estados, pero la salida puede cambiar entre flancos de reloj.

En este trabajo el receptor y el transmisor se implementaron como máquinas de Moore, ya que sus salidas van a la línea serie o son pulsos de aviso que deben ser limpios. El control del protocolo se implementó como Mealy, porque sus salidas (`rd` y `wr`) son señales internas que deben activarse en el mismo ciclo en que se consume o entrega un dato.

2.1.1 FSMD

Una FSM pura codifica toda su memoria en el registro de estado. Si el receptor se modelara así, necesitaría un estado distinto por cada combinación de "cuántos ticks pasaron", "cuántos bits llegaron" y "qué bits llegaron", lo que daría una cantidad inmanejable de estados.

La solución es separar el problema en dos partes, lo que se conoce como **FSMD** (*FSM with Datapath*):

- la **FSM** decide *qué hacer* y recuerda solo la fase (`IDLE`, `START`, `DATA`, ...);
- el **datapath** son registros auxiliares (contadores y registros de desplazamiento) que guardan *cuánto* y *con qué* se está trabajando.

Gracias a esta separación, el receptor se resuelve con cinco estados en lugar de miles.

2.1.2 Codificación de estados

Los estados se codificaron con **one-hot**, es decir un bit por estado, que es la codificación recomendada en FPGA: la lógica de decodificación se reduce a mirar un solo bit, lo que favorece la frecuencia máxima. Todas las máquinas incluyen una rama `default` que, ante una codificación inválida, devuelve la máquina a su estado inicial (máquina de estado segura).

2.2 Comunicación serie asíncrona

En una comunicación *asíncrona* no existe una línea de reloj compartida entre emisor y receptor. Ambos extremos solo acuerdan de antemano la **velocidad** en baudios, es decir la duración de cada bit. A 9600 baudios cada bit dura $1/9600 \approx 104{,}17\,\mu$s.

La información viaja agrupada en tramas con la siguiente estructura:

Campo	**Duración**	**Valor**
Línea en reposo	indefinida	1
Bit de start	1 bit	0
Bits de datos	8 bits	el byte, LSB primero
Bit de stop	1 bit	1

Table 1: Trama utilizada en este trabajo (8-N-1).

El bit de start es el que permite al receptor sincronizarse: su flanco de bajada marca el comienzo de la trama. A partir de ahí el receptor mide el tiempo por su cuenta, contando con su propio reloj.

2.2.1 Sobremuestreo 16x

Como el receptor solo conoce el flanco inicial, necesita una forma de ubicarse en el *centro* de cada bit, que es donde la señal está estable. Para eso se divide cada bit en 16 intervalos iguales, llamados *ticks*. Contando 8 ticks desde el flanco de start el receptor queda en el medio del bit de start, y a partir de ahí cada 16 ticks cae en el centro de cada bit de datos.

El factor 16 es interno del receptor: no se transmite ni el emisor lo conoce. Además del alineamiento, el sobremuestreo aporta tolerancia frente a la diferencia de frecuencia entre los relojes de ambos extremos.

3 Arquitectura del sistema

El diseño se organizó en tres bloques, siguiendo el diagrama de bloques propuesto en la consigna.

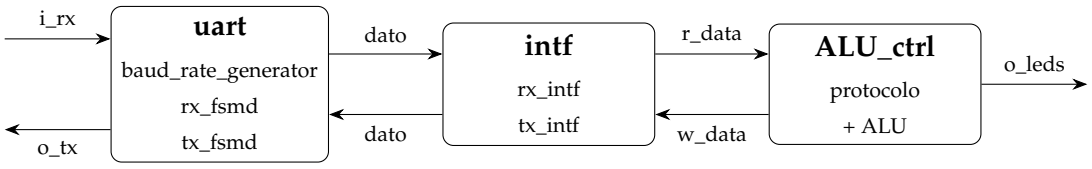


Figure 1: Bloques principales del sistema.

Módulo	Tipo	Función
`uart_alu_top`	top	instancia los tres bloques, expone los pines
`uart`	wrapper	agrupa el generador y las dos FSMD
`baud_rate_generator`	contador	genera el tick de sobremuestreo
`rx_fsmd`	FSMD	recibe una trama y entrega un byte
`tx_fsmd`	FSMD	transmite un byte como trama
`intf`	wrapper	agrupa los dos buffers
`rx_intf`	buffer	guarda el byte recibido, bandera `rx_empty`
`tx_intf`	buffer	bandera `tx_full` y pulso de arranque
`ALU_ctrl`	FSM	protocolo de tres bytes y registros A, B, opcode
`ALU`	combinacional	operaciones aritméticas y lógicas

Table 2: Módulos del diseño.

El criterio de separación fue que cada módulo tenga una única responsabilidad y una interfaz angosta. En particular, la UART no sabe qué significan los bytes que mueve, y el módulo `ALU_ctrl` es el único que conoce el protocolo. La ALU quedó como lógica puramente combinacional, sin reloj ni registros, tal como fue escrita en el trabajo práctico anterior, cambiando únicamente quién le entrega los operandos.

3.1 Esquemático

A continuación se incluyen los esquemáticos generados por Vivado luego de la elaboración. La Figura 2 muestra el módulo top con los tres bloques principales y sus conexiones; las Figuras 3 y 4 muestran el contenido de `uart` y de `intf` expandidos.

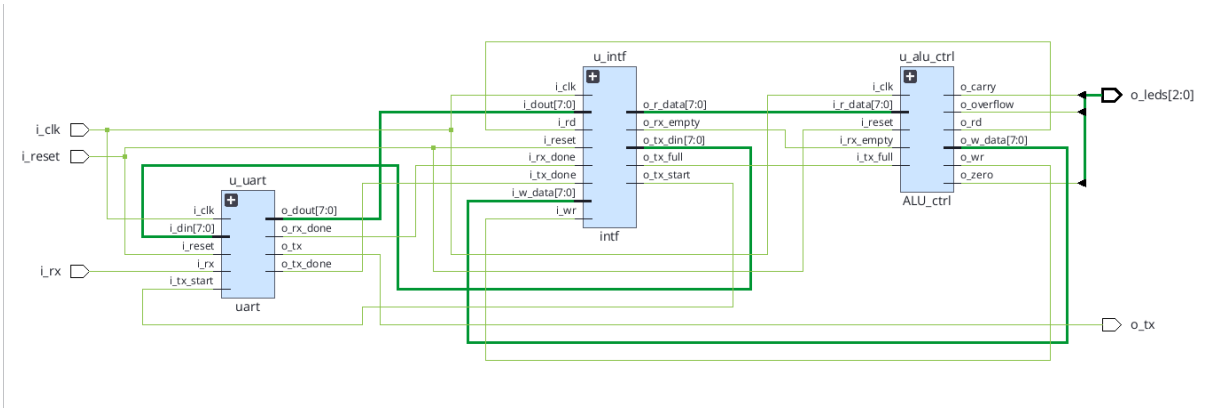


Figure 2: Esquemático del módulo top (`uart_alu_top`).

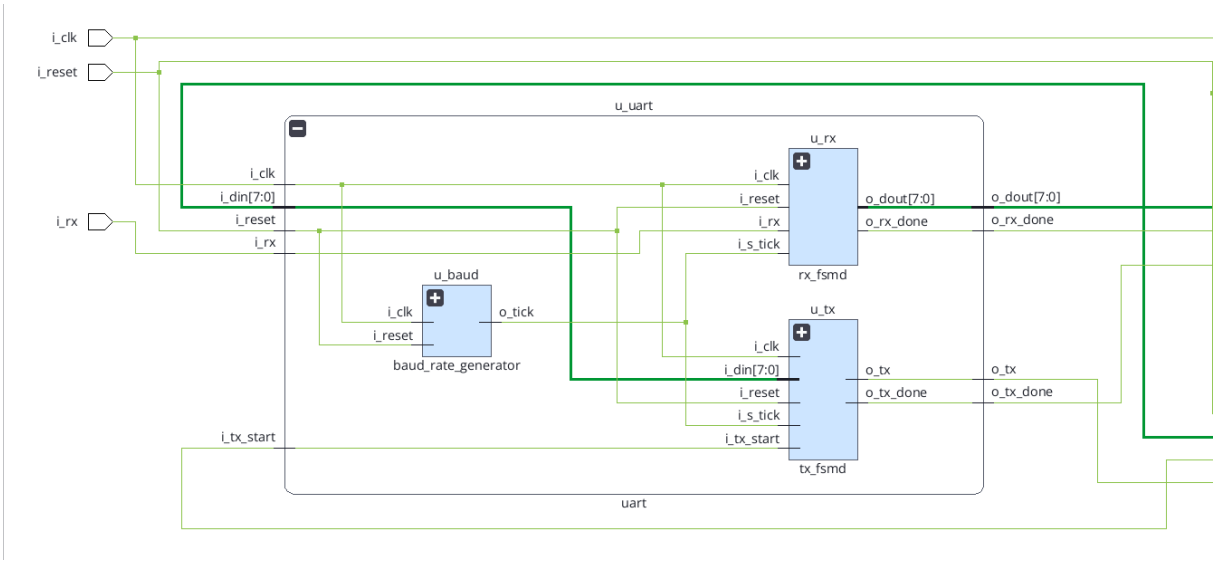


Figure 3: Módulo `uart` expandido: generador de baud rate, receptor y transmisor.

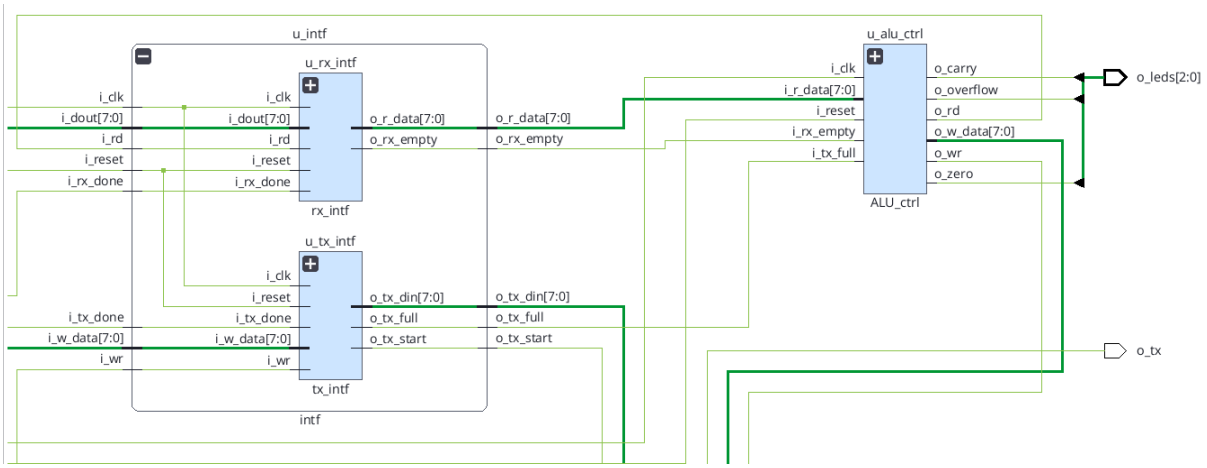


Figure 4: Módulo `intf` expandido (`rx_intf` y `tx_intf`) junto al control `ALU_ctrl`.

4 Generador de baud rate

Es un contador módulo N que emite un pulso de *un solo ciclo* de reloj cada N ciclos. Es importante destacar que **no es un reloj**: es una señal de habilitación (*enable*) que se usa como condición dentro de los bloques secuenciales. Utilizarla como reloj generaría un reloj derivado de lógica combinacional, algo desaconsejado en FPGA porque no utiliza la red dedicada de distribución de reloj y complica el análisis de tiempos.

El divisor se calcula como:

$$N = \frac{f_{clk}}{\text{BaudRate} \times 16} = \frac{100 \times 10^6}{9600 \times 16} = 651{,}04 \approx 651 \tag{1}$$

Baudios	N	**Bits del contador**	**Error**
9600	651	10	$+0{,}006\%$
19200	326	9	$-0{,}147\%$
38400	163	8	$-0{,}147\%$
115200	54	6	$+0{,}47\%$

Table 3: Divisor y error para distintas velocidades con reloj de 100 MHz.

Se eligió 9600 baudios porque el redondeo del divisor introduce el menor error de todas las opciones y la velocidad es más que suficiente para transmitir tres bytes por operación. El ancho del contador se calcula automáticamente con `$clog2`, de modo que al cambiar la velocidad no haya que ajustarlo a mano.

```
localparam integer N = CLK_FREQ / (BAUD_RATE * OVERSAMP);
localparam integer WIDTH = $clog2(N);

reg [WIDTH-1:0] counter = {WIDTH{1'b0}};

always @(posedge i_clk) begin
    if (i_reset) counter <= {WIDTH{1'b0}};
    else if (counter == N-1) counter <= {WIDTH{1'b0}};
    else counter <= counter + 1'b1;
end

assign o_tick = (counter == N-1);
```

Código 1: Generador de baud rate.

La salida se decodifica directamente del registro (*output equals state*), por lo que el pulso de un ciclo se obtiene sin lógica adicional.

5 Receptor

5.1 Máquina de estados (`rx_fsmd`)

El receptor es una FSMD con cinco estados. Su datapath está formado por tres registros:

- `s`: cuenta los ticks dentro del bit actual (0 a 15).
- `n`: cuenta los bits de datos ya recibidos.
- `b`: registro de desplazamiento donde se arma el byte.

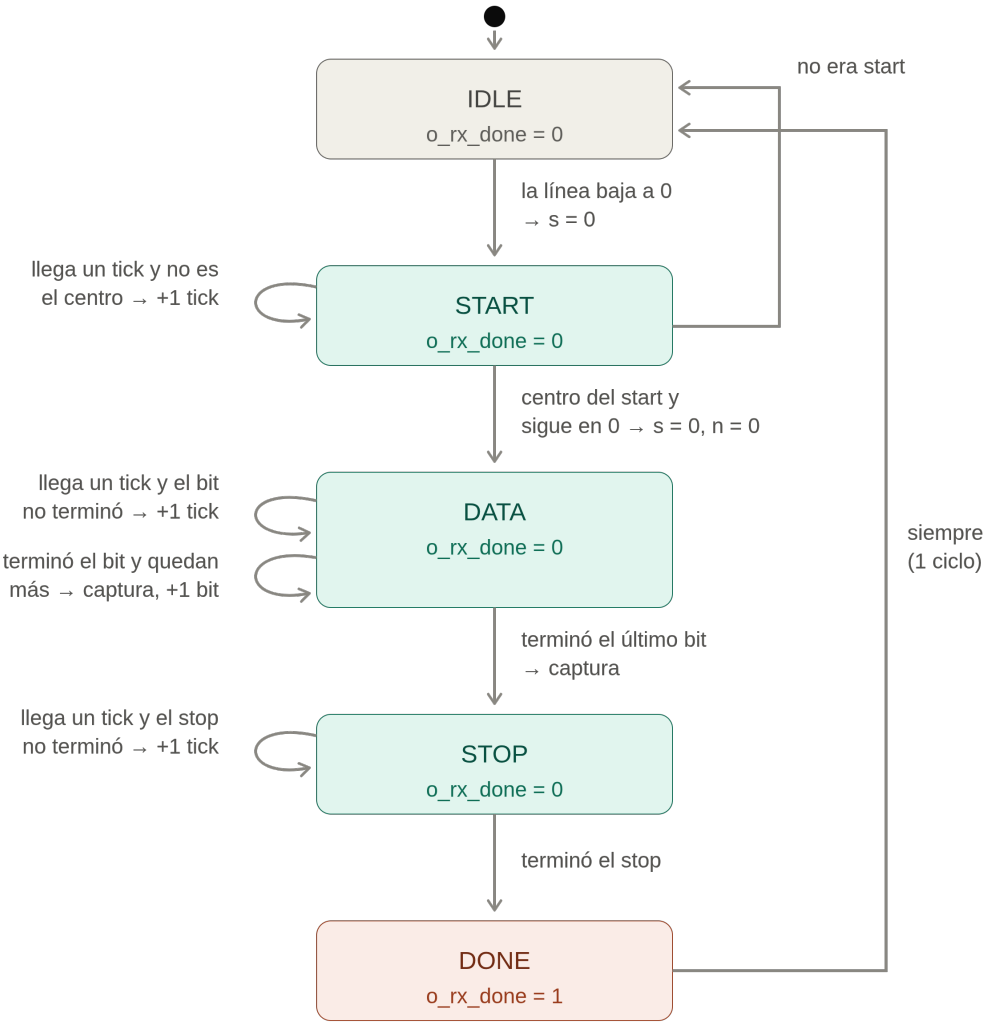


Figure 5: Diagrama de estados del receptor (máquina de Moore).

El funcionamiento estado por estado es el siguiente:

- **IDLE:** observa la línea en cada ciclo de reloj (no espera al tick, para no llegar tarde al flanco). Cuando la línea baja a 0, comienza una trama.

- **START:** cuenta 8 ticks, es decir medio bit, para ubicarse en el centro del bit de start. Si en ese punto la línea volvió a 1, lo que bajó fue ruido y la máquina regresa a `IDLE`.
- **DATA:** cada 16 ticks cae en el centro de un bit, lo muestrea y lo ingresa al registro de desplazamiento. Como UART transmite el LSB primero, el bit nuevo entra por la izquierda y el registro se desplaza a la derecha, de modo que al recibir los ocho bits el byte queda ordenado.
- **STOP:** espera un bit completo desde el centro del último dato, con lo cual termina en el centro del bit de stop.
- **DONE:** dura un solo ciclo y es el único estado donde `o_rx_done` vale 1. Existe para que ese aviso sea una salida de Moore.

```
DATA:
    if (i_s_tick) begin
        if (s == 15) begin
            s_next = {S_BITS{1'b0}};
            b_next = {rx_sync, b[DATA_BITS-1:1]};
            if (n == DATA_BITS-1)
                next_state = STOP;
            else
                n_next = n + 1'b1;
        end
        else
            s_next = s + 1'b1;
    end
```

Código 2: Estado DATA del receptor.

5.1.1 Sincronizador de entrada

La línea `i_rx` proviene de la PC y no guarda ninguna relación con el reloj de la FPGA, por lo que sus flancos pueden caer justo sobre un flanco de reloj y dejar un flip-flop en estado metaestable. Para evitarlo, la señal pasa por dos flip-flops en cascada antes de entrar a la lógica: el primero absorbe el riesgo y el segundo, un ciclo después, entrega un valor ya estable que todos los registros de la máquina leen por igual. Ambos se inicializan en 1, que es el valor de reposo de la línea, para no detectar un bit de start falso al encender la placa.

5.1.2 Elección del punto de muestreo

Una decisión de diseño importante fue terminar el estado `STOP` en el *centro* del bit de stop y no en su final. De esta manera queda medio bit de margen antes de que pueda comenzar la trama siguiente, lo que absorbe la diferencia de frecuencia acumulada entre los dos relojes a lo largo de la trama. En las simulaciones se verificó que con este criterio el receptor tolera desvíos de hasta un 3% con tramas consecutivas sin pausa.

5.2 Interface del receptor (`rx_intf`)

La FSMD muestra el byte recibido durante un único ciclo, junto con el pulso `o_rx_done`, y vuelve inmediatamente a esperar la trama siguiente. Esa ventana es demasiado

angosta para el resto del sistema, de modo que el interface guarda el dato y levanta una bandera que se mantiene hasta que alguien lo consuma.

```
wire accept = i_rx_done && (!full || i_rd);

always @(posedge i_clk) begin
    if (i_reset) full <= 1'b0;
    else if (accept) full <= 1'b1; // entro un byte
    else if (i_rd) full <= 1'b0; // lo consumieron
end

assign o_r_data = buffer;
assign o_rx_empty = ~full;
```

Código 3: Lógica del buffer de recepción.

La condición `accept` contempla el caso en que llega un byte nuevo en el mismo ciclo en que el consumidor lee el anterior, de modo que no se pierde ningún dato.

6 Transmisor

6.1 Máquina de estados (`tx_fsmd`)

Para el transmisor se optó por armar la **trama completa** en un único registro de desplazamiento al momento de recibir el pedido, en lugar de generar cada campo desde un estado distinto. Como start, datos y stop pasan a ser simplemente "un bit más de la trama", la máquina se reduce a tres estados.

La trama se arma con la concatenación `{stop, dato, start}`: en Verilog lo que se escribe a la izquierda ocupa los bits más significativos, por lo que el bit de start queda en la posición 0 y es el primero en salir.

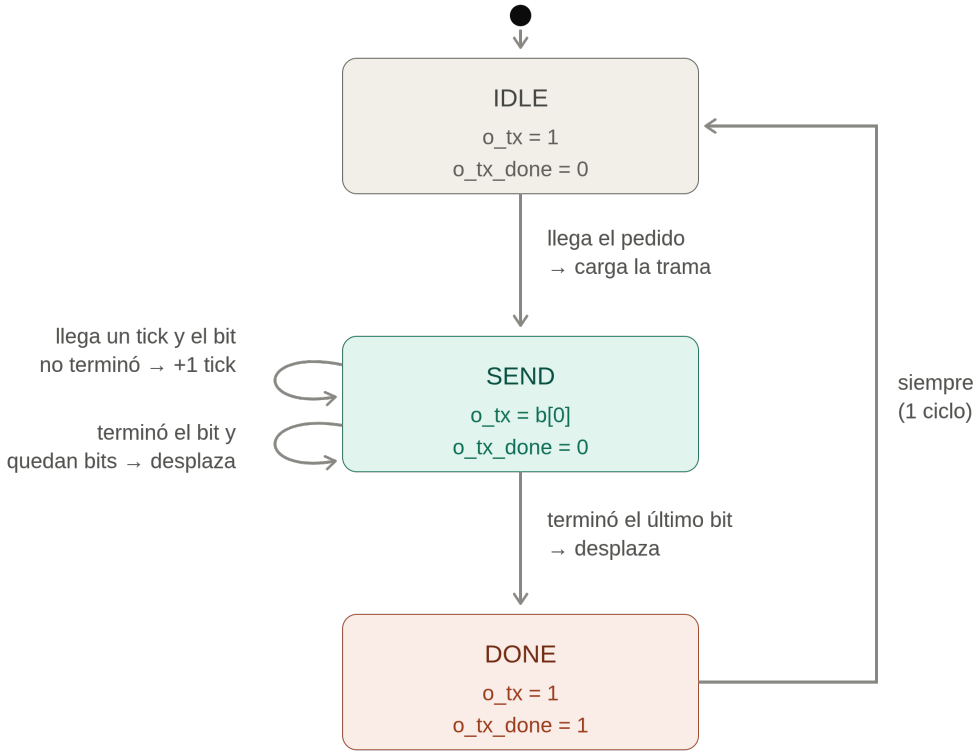


Figure 6: Diagrama de estados del transmisor (máquina de Moore).

```
IDLE:
    if (i_tx_start) begin
        b_next = {{STOP_BITS{STOP_BIT}}, i_din, START_BIT};
        s_next = {S_BITS{1'b0}};
        n_next = {N_BITS{1'b0}};
        next_state = SEND;
    end

SEND:
    if (i_s_tick) begin
        if (s == 15) begin
            s_next = {S_BITS{1'b0}};
            b_next = {1'b1, b[FRAME_BITS-1:1]};
            ...
```

Código 4: Carga de la trama y desplazamiento.

Al desplazar se introducen unos por la izquierda, de modo que cuando la trama termina de salir el registro queda en todos unos y la línea vuelve sola al estado de reposo. La salida es directamente un bit del registro (`o_tx = b[0]`), sin lógica intermedia, lo que garantiza una señal libre de *glitches*: esto es importante porque la línea sale del chip hacia un receptor asincrónico, que podría interpretar un pulso espurio como un bit de start.

Una diferencia con el receptor es que el estado `START` no existe como tal y que la trama se transmite completa, incluyendo el bit de stop entero: el transmisor debe garantizar la duración que promete, mientras que al receptor le conviene cortar antes para ganar margen.

6.2 Interface del transmisor (`tx_intf`)

A diferencia del interface de recepción, este no necesita guardar el dato, porque la FSMD lo copia a su propio registro en el mismo ciclo en que recibe el pedido. Lo único que almacena es *estado*: si hay una transmisión en curso.

```
always @(posedge i_clk) begin
    if (i_reset) full <= 1'b0;
    else if (i_wr && !full) full <= 1'b1; // arranca una transmision
    else if (i_tx_done) full <= 1'b0; // el tx_fsmd termino
end

assign o_tx_start = i_wr && !full;
assign o_tx_din = i_w_data;
assign o_tx_full = full;
```

Código 5: Bandera de ocupado del transmisor.

La bandera cumple dos funciones con una sola señal: le informa al control si puede escribir y, a la vez, impide que se genere un pulso de arranque mientras el transmisor está ocupado. Esto último es necesario porque la FSMD solo atiende el pedido cuando se encuentra en `IDLE`; un pulso emitido en cualquier otro momento se perdería sin aviso.

7 ALU y control del protocolo

7.1 Protocolo

El protocolo es posicional: la PC envía tres bytes y el orden determina su significado. No hay delimitadores ni bytes de sincronismo.

Byte	Sentido	Contenido
1	PC → FPGA	operando A
2	PC → FPGA	operando B
3	PC → FPGA	código de operación (se usan los 6 bits bajos)
4	FPGA → PC	resultado

Table 4: Secuencia de bytes de una operación.

Operación	Código	Operación	Código
ADD	`100000`	NOR	`100111`
SUB	`100010`	SRA	`000011`
AND	`100100`	SRL	`000010`
OR	`100101`	otro	resultado 0
XOR	`100110`		

Table 5: Códigos de operación de la ALU.

7.2 Máquina de control (`ALU_ctrl`)

Los tres primeros estados son idénticos entre sí salvo por el registro en el que guardan el byte: existen únicamente para recordar cuántos bytes llegaron. Al ser una máquina de Mealy, las salidas se indican sobre las transiciones.

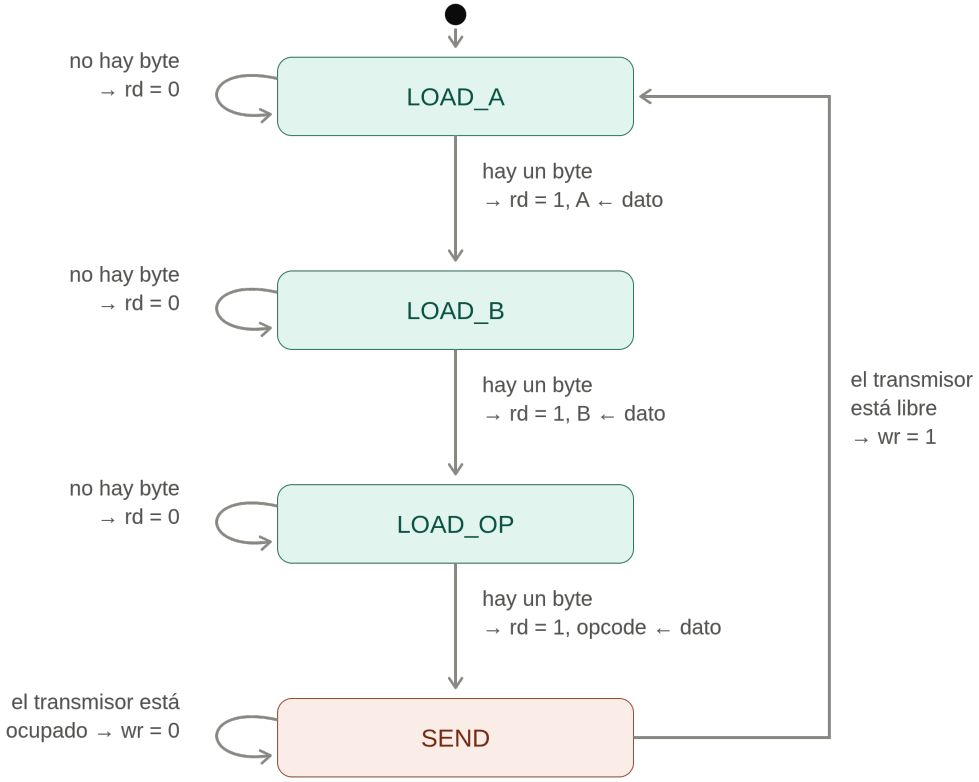


Figure 7: Diagrama de estados del control del protocolo (máquina de Mealy).

No fue necesario un estado de cálculo entre `LOAD_OP` y `SEND`: como la ALU es combinacional, en el ciclo siguiente a la carga del opcode el resultado ya está disponible en su salida. Tampoco hizo falta un estado de espera posterior al envío, porque el interface de transmisión retiene el dato y la condición `tx_full` impide escribir sobre una transmisión en curso.

7.3 ALU

La ALU se mantuvo tal como fue escrita en el trabajo anterior, con la única modificación de separarla de la lógica que capturaba los operandos desde los switches. Es puramente combinacional: no tiene reloj ni registros, y el resultado aparece apenas se estabilizan sus entradas. Sus banderas de estado se conectan a los LEDs de la placa.

8 Verificación

Se escribieron dos *testbenches* autoverificables, que informan por consola la cantidad de errores detectados.

8.1 Loopback del transmisor y el receptor

El primero conecta directamente la salida del transmisor con la entrada del receptor, de modo que todo lo transmitido debe volver idéntico. Verifica la cadena completa (generador de baud rate, armado de la trama y muestreo) sin depender de la placa.

Los patrones de prueba se eligieron para cubrir los casos límite: `0x55` y `0xAA` (bits alternados, sensibles a errores de alineación), `0x00` (nueve bits consecutivos en 0 contando el start), `0xFF` (datos indistinguibles de la línea en reposo) y `0x01` y `0x80` (verifican el orden LSB primero). El testbench también comprueba que la línea permanezca en 1 mientras el transmisor está en reposo.

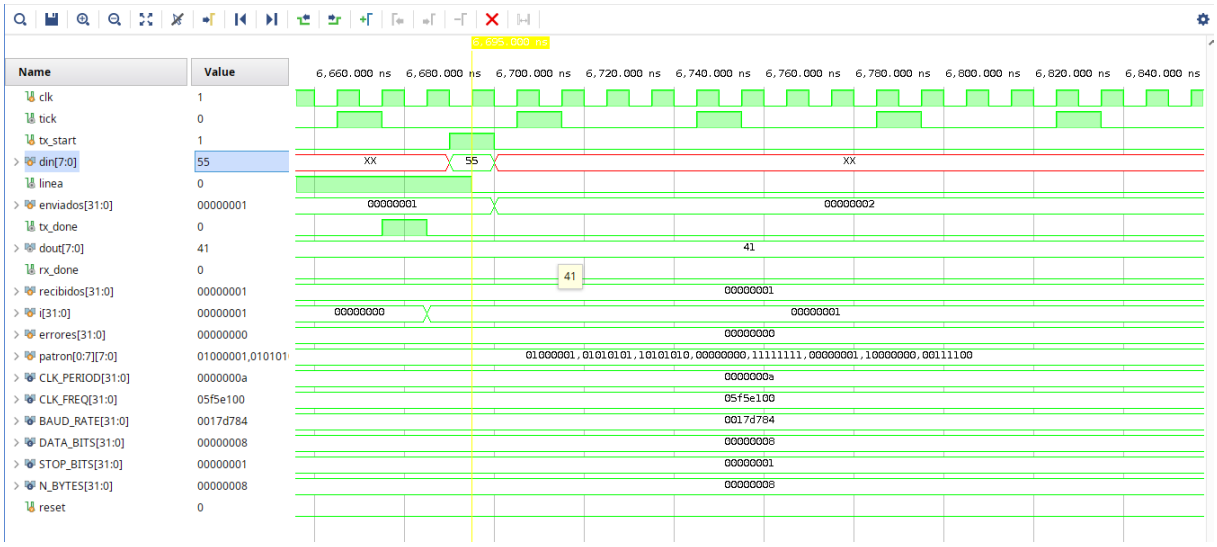


Figure 8: Formas de onda del testbench de loopback.

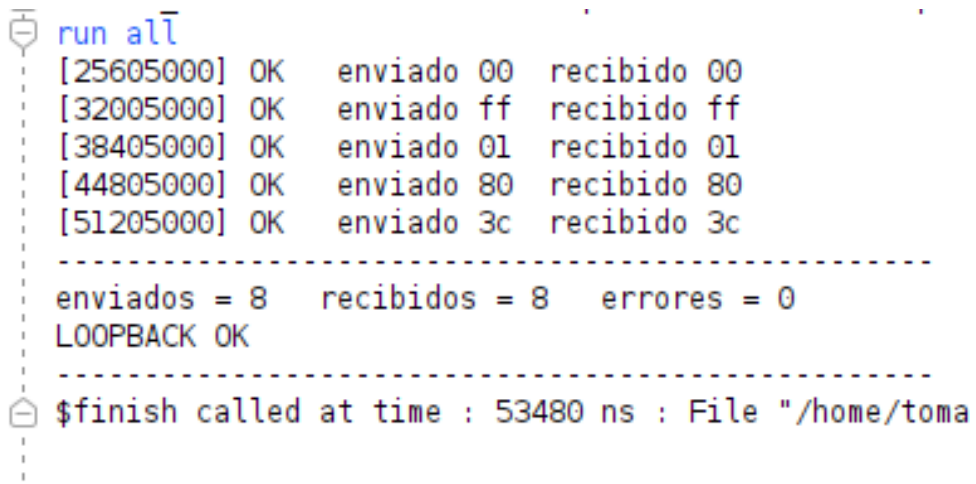


Figure 9: Salida por consola del testbench de loopback.

8.2 Sistema completo

El segundo testbench instancia el módulo top completo y, del otro lado de las líneas serie, una segunda instancia del módulo `uart` que cumple el rol de la PC: envía los tres bytes y recibe la respuesta. Se verificaron 16 casos, que incluyen las ocho operaciones y situaciones particulares.

Caso	A	B	Resultado	Banderas
ADD	05	03	08	–
SUB	03	05	FE	carry
AND	0F	F0	00	zero
OR	0F	F0	FF	–
XOR	0F	FF	F0	–
NOR	0F	F0	00	zero
SRA	80	01	C0	–
SRL	80	01	40	–
ADD con carry	FF	01	00	zero, carry
ADD con overflow	7F	01	80	overflow
Opcode inválido	09	02	00	zero

Table 6: Casos verificados en simulación (todos correctos).

Además se comprobó que por cada operación llegue exactamente un byte de respuesta, y que dos operaciones consecutivas sin pausa entre tramas se resuelvan correctamente.

Para acortar los tiempos de simulación se utilizó una velocidad elevada (`BAUD_RATE = 1562500`, que da $N = 4$); la lógica ejercitada es la misma, ya que lo único que cambia es cada cuántos ciclos llega el tick.

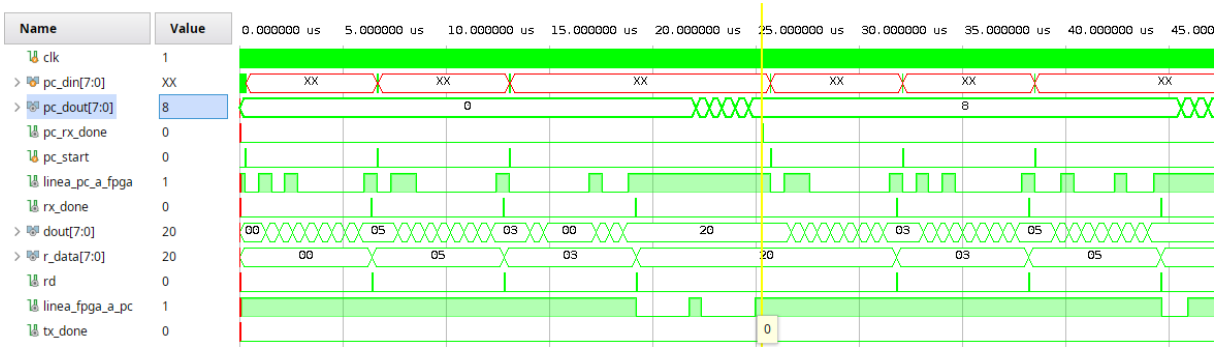


Figure 10: Formas de onda del testbench del sistema completo para una operación.

```
run all
OK    ADD  5+3: 05 , 03 -> 08   [z=0 c=0 v=0]
OK    SUB  3-5: 03 , 05 -> fe   [z=0 c=1 v=0]
OK    AND: 0f , f0 -> 00   [z=1 c=0 v=0]
OK    OR: 0f , f0 -> ff   [z=0 c=0 v=0]
OK    XOR: 0f , ff -> f0   [z=0 c=0 v=0]
OK    NOR: 0f , f0 -> 00   [z=1 c=0 v=0]
OK    SRA  negativo: 80 , 01 -> c0   [z=0 c=0 v=0]
OK    SRL  negativo: 80 , 01 -> 40   [z=0 c=0 v=0]
OK    ADD  carry: ff , 01 -> 00   [z=1 c=1 v=0]
OK    ADD  overflow: 7f , 01 -> 80   [z=0 c=0 v=1]
OK    SUB  zero: 07 , 07 -> 00   [z=1 c=0 v=0]
OK    ADD  ceros: 00 , 00 -> 00   [z=1 c=0 v=0]
OK    OR   patron: aa , 55 -> ff   [z=0 c=0 v=0]
OK    opcode invalido: 09 , 02 -> 00   [z=1 c=0 v=0]
OK    ADD  seguidas 1: 0a , 14 -> 1e   [z=0 c=0 v=0]
OK    ADD  seguidas 2: 14 , 0a -> 0a   [z=0 c=0 v=0]
-------------------------------------------------
pruebas = 16   errores = 0
SISTEMA OK
-------------------------------------------------
```

Figure 11: Salida por consola del testbench del sistema completo.

9 Implementación en la placa

9.1 Conexión física

La placa se conecta a la PC mediante un único cable USB que cumple tres funciones: alimenta la placa, permite programarla por JTAG y transporta la comunicación serie. La FPGA no maneja el protocolo USB: en la placa hay un chip puente (FT2232HQ) que traduce entre USB y UART, de modo que el diseño solo ve dos líneas serie comunes. Del lado de la PC, el sistema operativo crea un puerto serie virtual.

9.2 Asignación de pines

Puerto	Dirección	Pin	Función
`i_clk`	entrada	W5	oscilador de 100 MHz
`i_reset`	entrada	U18	pulsador central (btnC)
`i_rx`	entrada	B18	línea serie desde el puente
`o_tx`	salida	A18	línea serie hacia el puente
`o_leds[0]`	salida	U16	bandera zero
`o_leds[1]`	salida	E19	bandera carry
`o_leds[2]`	salida	U19	bandera overflow

Table 7: Asignación de pines (estándar LVCMOS33).

Un detalle a tener en cuenta es que, en el archivo de constraints provisto por Digilent, los nombres de los pines de la UART están puestos desde el punto de vista del chip puente: el pin B18, identificado como `RsRx`, corresponde a la *salida* del puente y por lo tanto es la entrada de la FPGA.

9.3 Prueba con la PC

Para operar el sistema se desarrolló un script de Python con interfaz gráfica. El script permite seleccionar el puerto y la velocidad, enviar los operandos y el código de operación, y muestra el resultado en hexadecimal, decimal (con y sin signo) y binario. Además incluye un modelo de la ALU escrito en Python, con el que compara automáticamente la respuesta de la FPGA e indica si coincide.

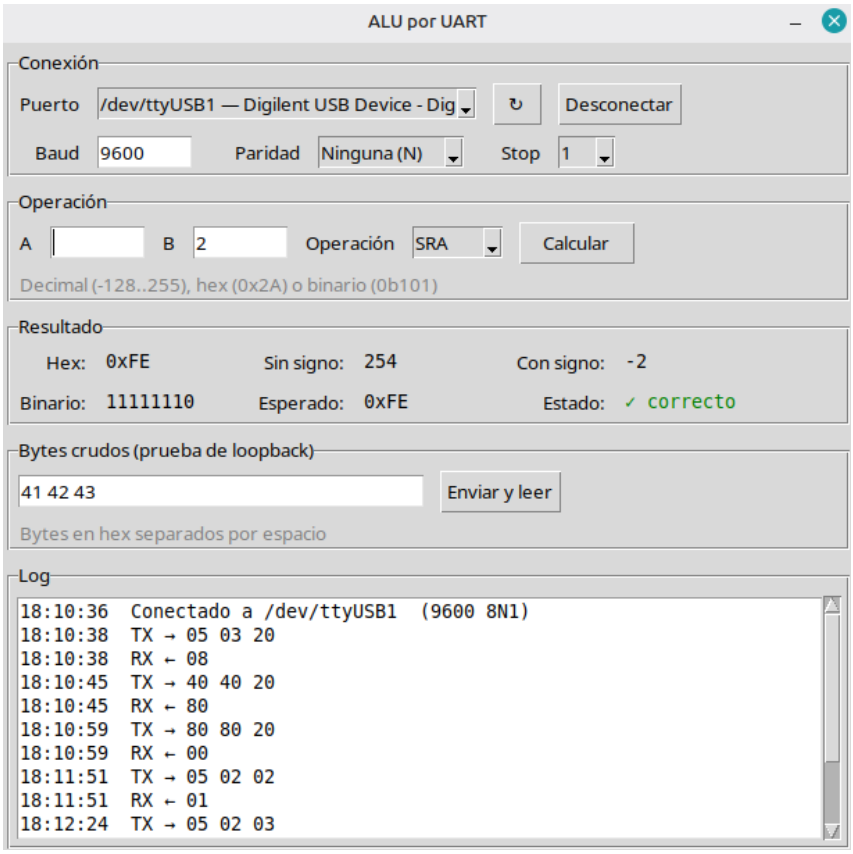


Figure 12: GUI del script de python utilizado

Una particularidad de la comunicación es que los valores deben enviarse como *bytes crudos* y no como caracteres: escribir "5" en una terminal convencional transmite el código ASCII correspondiente (`0x35`) y no el número 5. Esa es la razón por la que se optó por un script propio en lugar de una terminal serie genérica.

10 Conclusiones

Se logró implementar una UART funcional y utilizarla para operar la ALU desde una PC, cumpliendo con el objetivo del trabajo. El sistema fue verificado en simulación mediante dos testbenches autoverificables y luego probado sobre la placa.

Las principales conclusiones del desarrollo son:

- La separación entre FSM y datapath (FSMD) es lo que hace manejable el problema: sin ella, el receptor requeriría una cantidad inviable de estados.
- La señal del generador de baud rate debe usarse como habilitación y nunca como reloj, para no generar relojes derivados de lógica.
- Toda señal proveniente del exterior necesita un sincronizador de dos flip-flops antes de entrar a la lógica, ya que de lo contrario distintos registros pueden leer valores distintos en el mismo ciclo.
- El patrón de *dato más bandera de validez* se repite en cada frontera del sistema y es lo que permite que módulos con tiempos muy distintos (una trama dura unos 10^5 ciclos de reloj, un pulso dura uno) trabajen juntos sin perder información.
- Definir interfaces angostas y estables entre módulos permitió reutilizar la ALU del trabajo anterior sin modificar una sola línea de su lógica.

Como posibles mejoras quedan pendientes el agregado de bit de paridad y detección de errores de trama, y un protocolo con byte de sincronismo: al ser posicional, si se pierde o se cuela un byte la máquina de control queda desfasada y es necesario reiniciarla con el pulsador de reset.

References

[1] P. P. Chu, *FPGA Prototyping by Verilog Examples: Xilinx Spartan-3 Version*. Wiley, 2008.

[2] Digilent, *Basys 3 FPGA Board Reference Manual*, 2016.

[3] Digilent, *Basys-3-Master.xdc*, repositorio `digilent-xdc`, `https://github.com/Digilent/digilent-xdc`.

[4] Cátedra de Arquitectura de Computadoras, *Máquinas de Estado Finitas – TP: UART*, FCEFyN, UNC.